

Nat Curtis

ME 3313

9/10/2025

ME 3313 Honors Project

Introduction

For this project, I am analyzing the linkages of a skid-steer bucket loader, shown in Figure 1. I will explore each of the topics we explore in ME 3313, starting with mobility, and going on to topics including range of motion, position, velocity, acceleration, and forces, and use each of these topics to analyze the linkage. I will then explore synthesis and try to create my own linkage for the bucket loader.



Figure 1: Side view of a Skid

Mobility

Mobility is a measure of how many degrees of freedom a linkage has, and it is calculated with Equation (1), called the Grubler-Kutzbach Mobility equation, where M is the mobility, L is the number of links, J_1 is the number of full joints, and J_2 is the number of half joints.

$$M = 3(L - 1) - 2J_1 - J_2 \quad (1)$$

A link is a single rigid body. A full joint is a joint with 1 degree of freedom, like a pin or two flat surfaces sliding along each other. A half joint is a joint with 2 degrees of freedom, like a pin in a joint, or two curved surface rolling or slipping along each other. On a kinematic diagram of a linkage, each link is marked with a unique plain number from 1 to the number of links, each full joint is marked with a unique circled number from 1 to the number of full joints, and each half joint is marked with a unique boxed number from 1 to the

number of half joints. As shown in Figure 2, the main linkage of the skid-steer has 6 links, 7 full joints, and 0 half joints. Using the mobility equation as shown in Equation (2), the mobility is calculated to be 1.

$$M = 3(L - 1) - 2J_1 - J_2 = 3(6 - 1) - 2 * 7 - 0 = 1 \quad (2)$$

Because the mobility is 1, only one input is needed to control the linkage. In this case, the input is a hydraulic piston, which is a combination of links 4 and 6. Figure 3 shows the annotated linkage diagram overlain on the original image of the skid-steer.

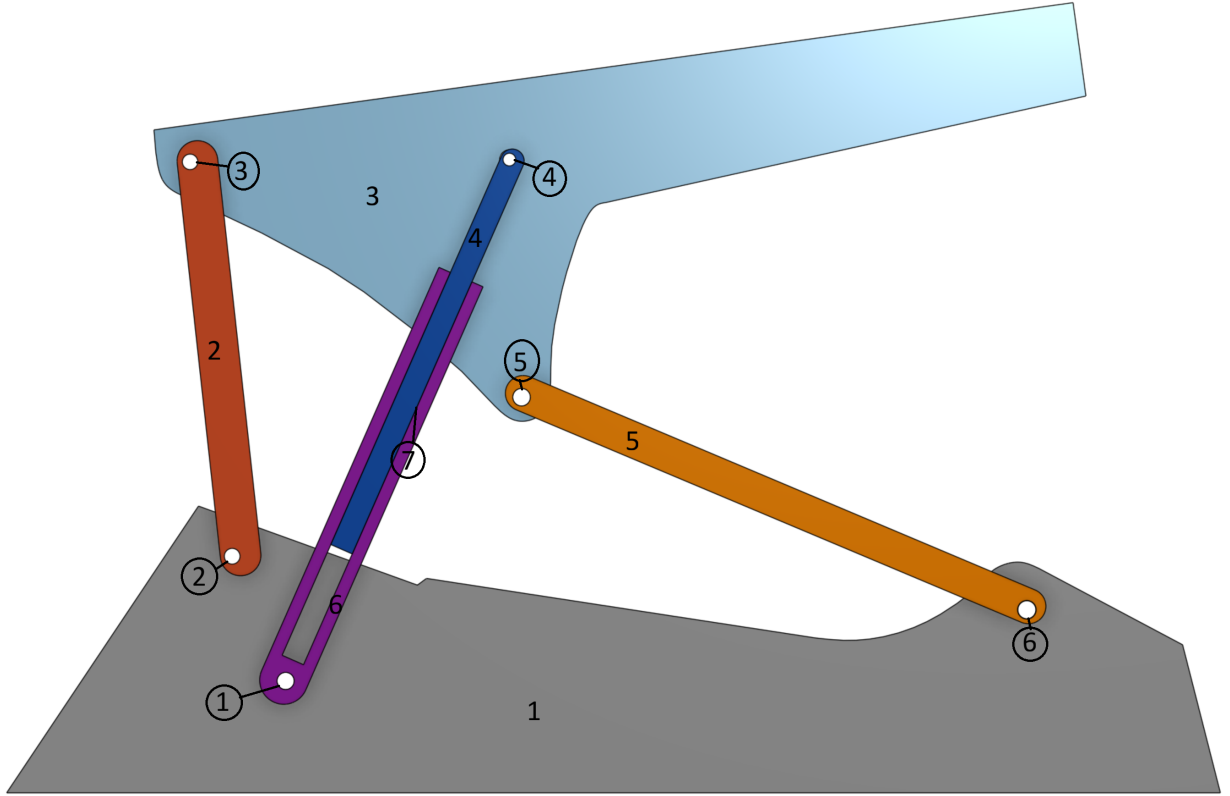


Figure 2: Annotated Linkage



Figure 3: Annotated Linkage Overlain On Skid-Steer

Multiloop Closure Equations

Mobility allows us to determine the degrees of freedom of a linkage, but usually we also need to determine the positions and angles of the individual links. To do this, we can use a method called Multiloop closure. It consists of drawing vectors between joints of each link, then writing equation summing loops of vectors, because the RHS of the equation has to equal zero. This is because the last vector ends where the first vector starts. These vector equations can then be turned into their scalar forms, which can be combined with equations relating rigid geometry of the links, providing a system of equations to solve. When done correctly, the number of unknowns minus the number of scalar and rigid geometry equations should equal the mobility. For the examined linkage, the vectors and loops are labeled as shown in Figure 4. The vector equations are shown below as eqs. (3) and (4).

$$\vec{R}_{BA} + \vec{R}_{DB} - \vec{R}_{CA} - \vec{R}_{GC} - \vec{R}_{DG} = 0 \quad (3)$$

$$\vec{R}_{GC} + \vec{R}_{DG} - \vec{R}_{DE} - \vec{R}_{EF} - \vec{R}_{FC} = 0 \quad (4)$$

These equations can be expanded into their scalar forms as shown in eqs. (5) to (10) below, where each θ is measured counter-clockwise from the horizontal to the tail of the corresponding vector.

$$R_{BA} \cos \theta_{BA} + R_{DB} \cos \theta_{DB} - R_{CA} \cos \theta_{CA} - R_{GC} \cos \theta_{GC} - R_{DG} \cos \theta_{DG} = 0 \quad (5)$$

$$R_{BA} \sin \theta_{BA} + R_{DB} \sin \theta_{DB} - R_{CA} \sin \theta_{CA} - R_{GC} \sin \theta_{GC} - R_{DG} \sin \theta_{DG} = 0 \quad (6)$$

$$R_{GC} \cos \theta_{GC} + R_{DG} \cos \theta_{DG} - R_{DE} \cos \theta_{DE} - R_{EF} \cos \theta_{EF} - R_{FC} \cos \theta_{FC} = 0 \quad (7)$$

$$R_{GC} \sin \theta_{GC} + R_{DG} \sin \theta_{DG} - R_{DE} \sin \theta_{DE} - R_{EF} \sin \theta_{EF} - R_{FC} \sin \theta_{FC} = 0 \quad (8)$$

$$\theta_{GC} - \theta_{DG} = 0 \quad (9)$$

$$\theta_{DB} + \beta_D - \theta_{DE} = 0 \quad (10)$$

In these equations, R_{BA} , R_{DB} , R_{CA} , θ_{CA} , R_{GC} , R_{DE} , R_{EF} , R_{FC} , θ_{FC} , and β_D are known from the geometry of the linkage and do not change. This leaves θ_{BA} , θ_{DB} , θ_{GC} , R_{DG} , θ_{DG} , θ_{DE} , and θ_{EF} as unknowns for a total of 7 unknowns. Taking 7 unknowns minus 6 equations yields 1 degree of freedom, which agrees with the earlier mobility analysis. Also, R_{DG} will be the input because it is controlled by a hydraulic piston.

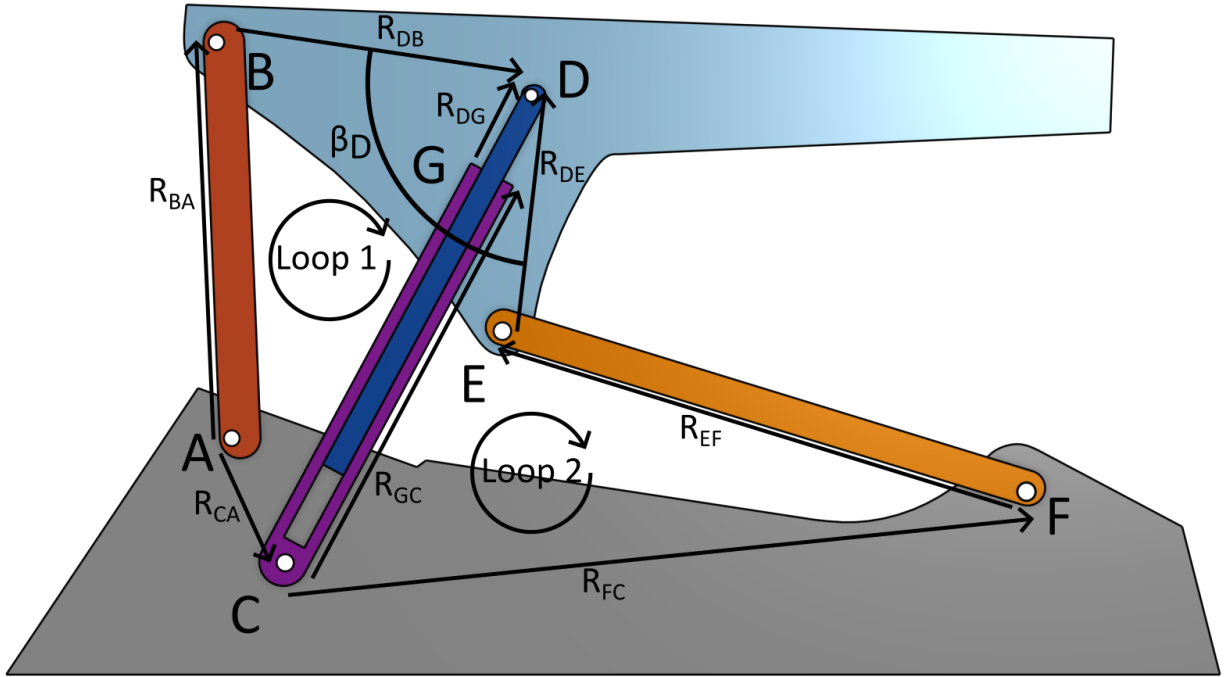


Figure 4: Diagram showing vector for Multiloop Equations

Loop Equation Derivatives

The next step in analyzing the linkage is to take the first and second derivatives of the scalar loop equations, which we can then use to find equations for velocity and acceleration of the links. The first derivatives of eqs. (5) to (10) are as shown below as eqs. (11) to (16), respectively.

$$-R_{BA}\omega_{BA} \sin \theta_{BA} - R_{DB}\omega_{DB} \sin \theta_{DB} + R_{GC}\omega_{GC} \sin \theta_{GC} - \dot{R}_{DG} \cos \theta_{DG} + R_{DG}\omega_{DG} \sin \theta_{DG} = 0 \quad (11)$$

$$R_{BA}\omega_{BA} \cos \theta_{BA} + R_{DB}\omega_{DB} \cos \theta_{DB} - R_{GC}\omega_{GC} \cos \theta_{GC} - \dot{R}_{DG} \sin \theta_{DG} - R_{DG}\omega_{DG} \cos \theta_{DG} = 0 \quad (12)$$

$$-R_{GC}\omega_{GC}\sin\theta_{GC} + \dot{R}_{DG}\cos\theta_{DG} - R_{DG}\omega_{DG}\sin\theta_{DG} + R_{DE}\omega_{DE}\sin\theta_{DE} + R_{EF}\omega_{EF}\sin\theta_{EF} = 0 \quad (13)$$

$$R_{GC}\omega_{GC}\cos\theta_{GC} + \dot{R}_{DG}\sin\theta_{DG} + R_{DG}\omega_{DG}\cos\theta_{DG} - R_{DE}\omega_{DE}\cos\theta_{DE} - R_{EF}\omega_{EF}\cos\theta_{EF} = 0 \quad (14)$$

$$\omega_{GC} - \omega_{DG} = 0 \quad (15)$$

$$\omega_{DB} - \omega_{DE} = 0 \quad (16)$$

The second derivatives of eqs. (5) to (10) are shown below as eqs. (17) to (22), respectively.

$$\begin{aligned} & -R_{BA}\alpha_{BA}\sin\theta_{BA} - R_{BA}\omega_{BA}^2\cos\theta_{BA} - R_{DB}\alpha_{DB}\sin\theta_{DB} \\ & - R_{DB}\omega_{DB}^2\cos\theta_{DB} + R_{GC}\alpha_{GC}\sin\theta_{GC} + R_{GC}\omega_{GC}^2\cos\theta_{GC} - \ddot{R}_{DG}\cos\theta_{DG} \\ & + 2\dot{R}_{DG}\omega_{DG}\sin\theta_{DG} + R_{DG}\alpha_{DG}\sin\theta_{DG} + R_{DG}\omega_{DG}^2\cos\theta_{DG} = 0 \end{aligned} \quad (17)$$

$$\begin{aligned} & R_{BA}\alpha_{BA}\cos\theta_{BA} - R_{BA}\omega_{BA}^2\sin\theta_{BA} + R_{DB}\alpha_{DB}\cos\theta_{DB} - R_{DB}\omega_{DB}^2\sin\theta_{DB} \\ & - R_{GC}\alpha_{GC}\cos\theta_{GC} + R_{GC}\omega_{GC}^2\sin\theta_{GC} - \ddot{R}_{DG}\sin\theta_{DG} \\ & - 2\dot{R}_{DG}\omega_{DG}\cos\theta_{DG} - R_{DG}\alpha_{DG}\cos\theta_{DG} + R_{DG}\omega_{DG}^2\sin\theta_{DG} = 0 \end{aligned} \quad (18)$$

$$\begin{aligned} & -R_{GC}\alpha_{GC}\sin\theta_{GC} - R_{GC}\omega_{GC}^2\cos\theta_{GC} + \ddot{R}_{DG}\cos\theta_{DG} - 2\dot{R}_{DG}\omega_{DG}\sin\theta_{DG} \\ & - R_{DG}\alpha_{DG}\sin\theta_{DG} - R_{DG}\omega_{DG}^2\cos\theta_{DG} + R_{DE}\alpha_{DE}\sin\theta_{DE} \\ & + R_{DE}\omega_{DE}^2\cos\theta_{DE} + R_{EF}\alpha_{EF}\sin\theta_{EF} + R_{EF}\omega_{EF}^2\cos\theta_{EF} = 0 \end{aligned} \quad (19)$$

$$\begin{aligned} & R_{GC}\alpha_{GC}\cos\theta_{GC} - R_{GC}\omega_{GC}^2\sin\theta_{GC} + \ddot{R}_{DG}\sin\theta_{DG} + 2\dot{R}_{DG}\omega_{DG}\cos\theta_{DG} \\ & + R_{DG}\alpha_{DG}\cos\theta_{DG} - R_{DG}\omega_{DG}^2\sin\theta_{DG} - R_{DE}\alpha_{DE}\cos\theta_{DE} \\ & + R_{DE}\omega_{DE}^2\sin\theta_{DE} - R_{EF}\alpha_{EF}\cos\theta_{EF} + R_{EF}\omega_{EF}^2\sin\theta_{EF} = 0 \end{aligned} \quad (20)$$

$$\alpha_{GC} - \alpha_{DG} = 0 \quad (21)$$

$$\alpha_{DB} - \alpha_{DE} = 0 \quad (22)$$

Code Introduction

To solve these equations, I used Python code taking advantage of the `fsolve` function from the SciPy library. This is a numerical solver function that takes in a set of equations and initial guesses and outputs a solution. Depending on the initial guess, this solution can be incorrect, or a valid solution but not the correct solution for the actual problem constraints. To increase the chances significantly of a valid solution being generated, I generated a range of initial condition options for each unknown variable and then generated a solution for every combination of initial conditions. Taking these, I kept only ones that had little error when plugged back into the multiloop equations and each variable was within an estimated range of motion. The known link lengths that were plugged into the equations are shown below, and the input length is shown under that. These measurements are only estimates, however, because I found them by finding the length of the loader in its documentation, then measured the length in a side view image of the loader. I took the ratio of these, then measured each link length and angle in the picture and multiplied each link length by the ratio to get the estimated actual link length. Angles however are not affected by scaling an image up or down though, so I used the measured angles as they were.

I then took this solution and used it to find solutions along a range of input angles. Because angles that are close together will have similar solutions, I was able to start at my calculated solution and use it as the initial guess for a slightly different angle. This would quickly converge to another solution close by, yielding a correct solution. Using this new solution as my next guess, I was able to repeat this and get the solution for a range of input angles. I then wrote equations for the absolute position of each joint in the linkage, with the origin at Joint C. The vector forms of these equations are shown below in eqs. (23) to (30). Using these equations and the range of solutions I found previously I plotted the path each joint followed for that range of angles, shown in Figure 5.

Next, I took the list of angles for each input length and used these angles and the previously measured link lengths in eqs. (11) to (22) to solve for angular velocity and angular acceleration of each point. The angular velocity and acceleration of Link BA is plotted in Figure 6 and Figure 7.

The next step is to calculate the accelerations of the centers of mass of each link. These are calculated using one equation for each, so can be solved directly without using a system of equations. These equations use relative accelerations with respect to points with known acceleration. The acceleration of Link BA can be seen in Figure 8.

The final part of the analysis is to write equations to solve for the forces and moments at each joint. The first step is to draw the free body diagrams of each link except for the ground link. Then using these, I wrote equations for the sum of forces and moments for each link. This gave me 15 equations and 15 unknowns for each angle once I set the output force. The output and input forces as well as the positions of the center of inertia of each link can be seen in Figure 9. These equations are all linear equations, so because there are the same number of unknowns as equations, I used the NumPy linear solver function to solve these equations at each position. The plots of forces and moments I found can be seen in figs. 10 to 18.

Vector Loop Equations for Joints

$$\vec{A} = -R_{CA} \vec{e}_1 \quad (23)$$

$$\vec{B} = -R_{CA} \vec{e}_1 + R_{BA} \vec{e}_2 \quad (24)$$

$$\vec{C} = 0 \quad (25)$$

$$\vec{D} = R_{FC} \vec{e}_1 + R_{EF} \vec{e}_2 + R_{DE} \vec{e}_3 \quad (26)$$

$$\vec{E} = R_{FC} \vec{r} + R_{EF} \vec{r} \quad (27)$$

$$\vec{F} = R_{FC} \vec{r} \quad (28)$$

$$\vec{G} = R_{GC} \vec{r} \quad (29)$$

$$\vec{P} = -R_{CA} \vec{r} + R_{BA} \vec{r} + R_{PB} \vec{r} \quad (30)$$

Link Plots

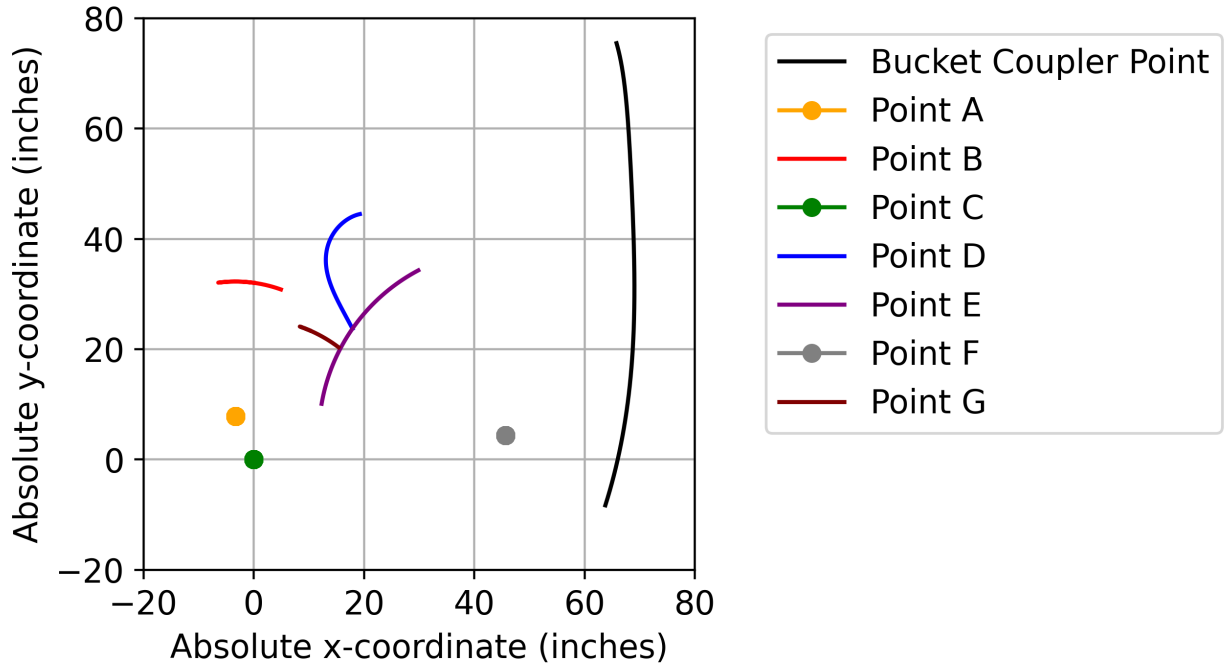


Figure 5: Path of absolute position of linkage joints

Next, I took the list of angles for each input length and used these angles and the previously measured link lengths in eqs. (11) to (22) to solve for angular velocity and angular acceleration of each point. The angular velocity and acceleration of Link BA is plotted in Figure 6 and Figure 7.

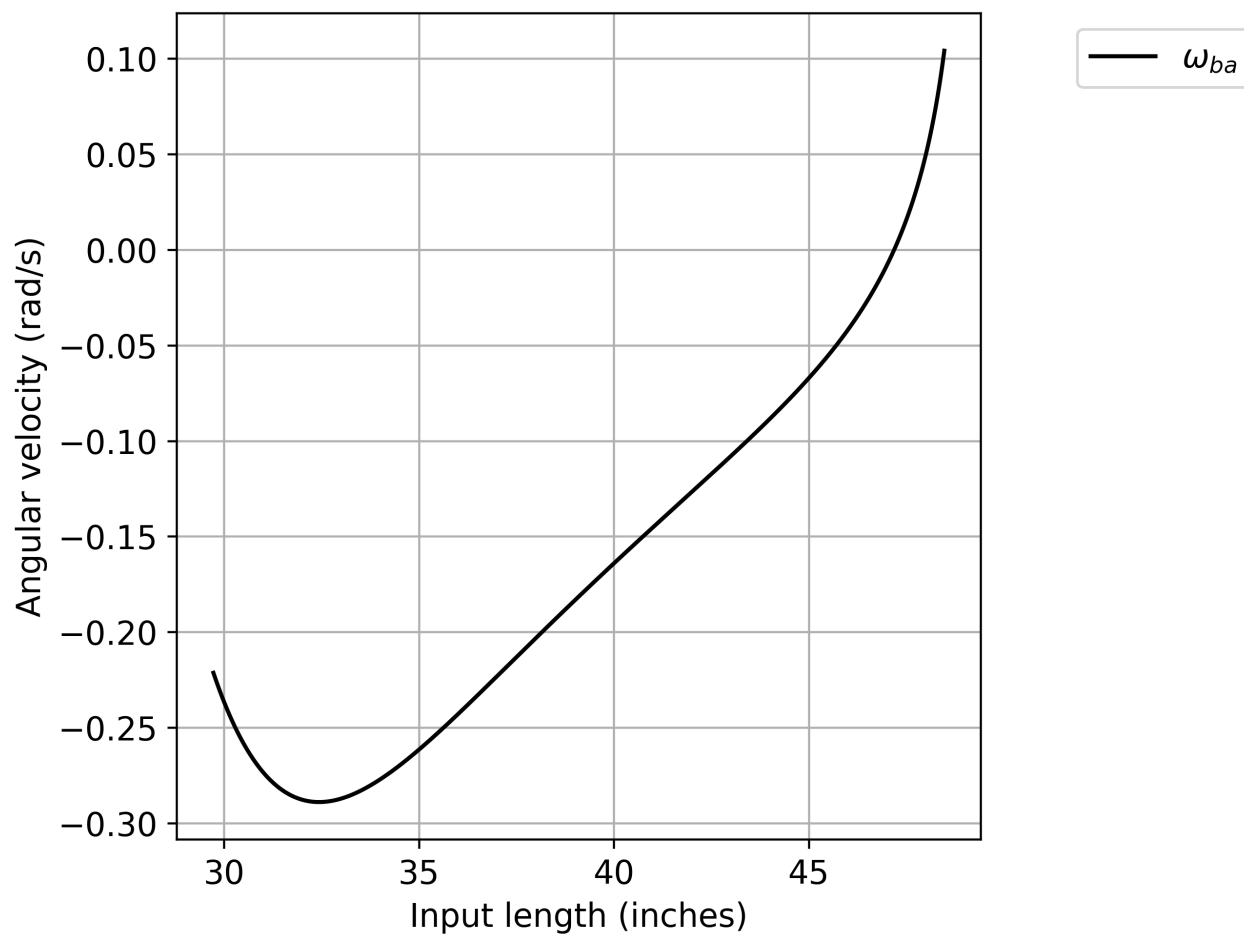


Figure 6: Angular velocity of Link BA vs the input length

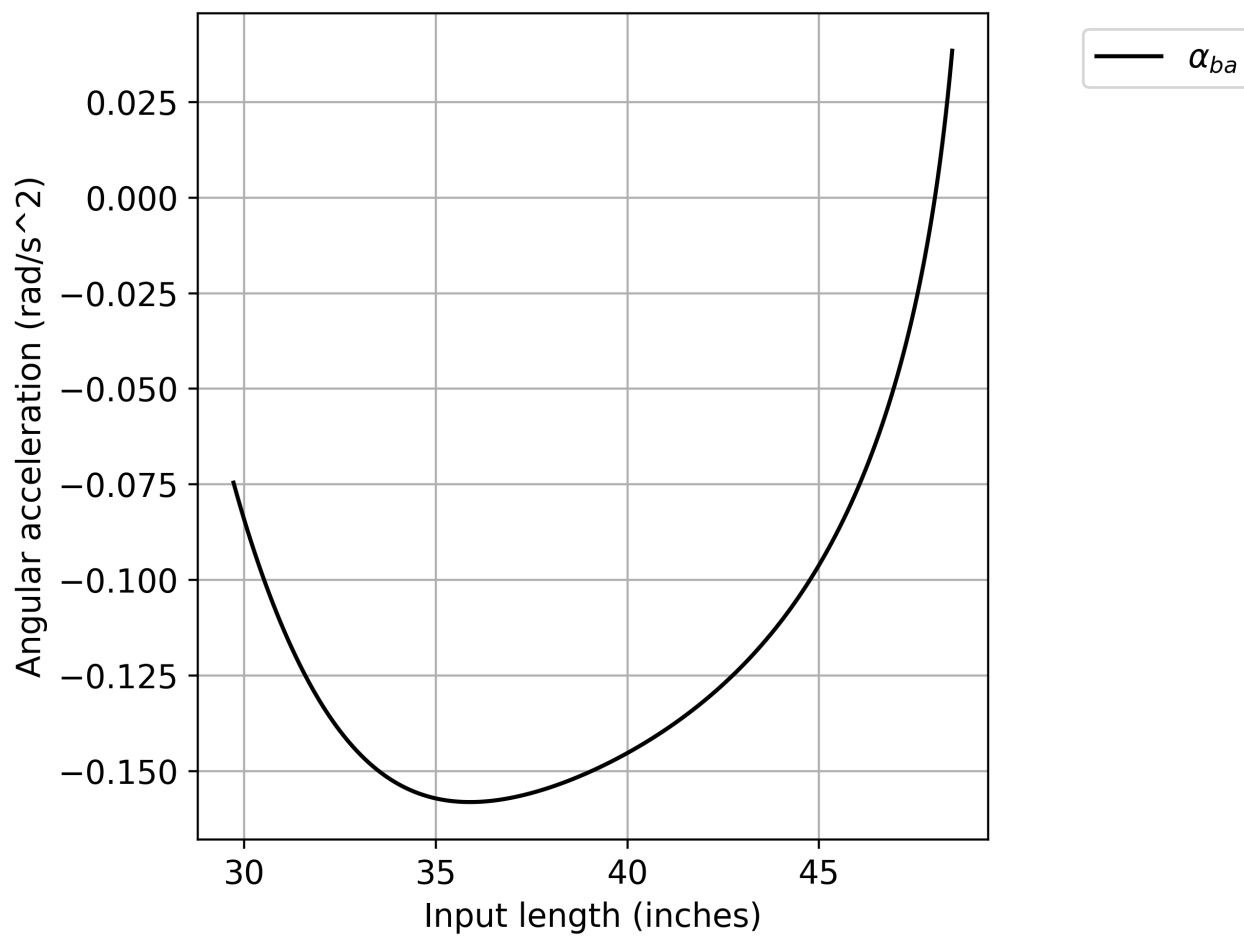


Figure 7: Angular acceleration of Link BA vs the input length

The next step is to calculate the accelerations of the centers of mass of each link. These are calculated using one equation for each, so can be solved directly without using a system of equations. These equations use relative accelerations with respect to points with known acceleration. These equations are shown below as Eqs. 31–40. The acceleration of Link BA can be seen in Figure 8.

Accelerations of Link Center of Gravities

$$A_{G2}^x = L_{BA} \cos(\theta_{BA} + \beta_{BA} + \pi) \omega_{BA}^2 + L_{BA} \cos(\theta_{BA} + \beta_{BA} + \pi/2) \alpha_{BA} \quad (31)$$

$$A_{G2}^y = L_{BA} \sin(\theta_{BA} + \beta_{BA} + \pi) \omega_{BA}^2 + L_{BA} \sin(\theta_{BA} + \beta_{BA} + \pi/2) \alpha_{BA} \quad (32)$$

$$A_{G3}^x = R_{BA} \cos(\theta_{BA} + \pi) \omega_{BA}^2 + R_{BA} \cos(\theta_{BA} + \pi/2) \alpha_{BA} \\ + L_{DB} \cos(\theta_{DB} + \beta_{DB} + \pi) \omega_{DB}^2 + L_{DB} \cos(\theta_{DB} + \beta_{DB} + \pi/2) \alpha_{DB} \quad (33)$$

$$A_{G3}^y = R_{BA} \sin(\theta_{BA} + \pi) \omega_{BA}^2 + R_{BA} \sin(\theta_{BA} + \pi/2) \alpha_{BA} \\ + L_{DB} \sin(\theta_{DB} + \beta_{DB} + \pi) \omega_{DB}^2 + L_{DB} \sin(\theta_{DB} + \beta_{DB} + \pi/2) \alpha_{DB} \quad (34)$$

$$A_{G4}^x = (R_{DC} - R_4) \cos(\theta_{GC} + \pi) \omega_{GC}^2 + (R_{DC} - R_4) \cos(\theta_{GC} + \pi/2) \alpha_{GC} \\ + L_4 \cos(\theta_{DG} + \beta_4 + \pi) \omega_{DG}^2 + L_4 \cos(\theta_{DG} + \beta_4 + \pi/2) \alpha_{DG} \\ + 2\dot{r}_{DG} \cos(\theta_{DG} + \beta_4 + 3\pi/2) \omega_{DG}^2 + \ddot{r}_{DG} \cos(\theta_{DG} + \beta_4) \quad (35)$$

$$A_{G4}^y = (R_{DC} - R_4) \sin(\theta_{GC} + \pi) \omega_{GC}^2 + (R_{DC} - R_4) \sin(\theta_{GC} + \pi/2) \alpha_{GC} \\ + L_4 \sin(\theta_{DG} + \beta_4 + \pi) \omega_{DG}^2 + L_4 \sin(\theta_{DG} + \beta_4 + \pi/2) \alpha_{DG} \\ + 2\dot{r}_{DG} \sin(\theta_{DG} + \beta_4 + 3\pi/2) \omega_{DG}^2 + \ddot{r}_{DG} \sin(\theta_{DG} + \beta_4) \quad (36)$$

$$A_{G5}^x = L_{EF} \cos(\theta_{EF} + \beta_{EF} + \pi) \omega_{EF}^2 + L_{EF} \cos(\theta_{EF} + \beta_{EF} + \pi/2) \alpha_{EF} \quad (37)$$

$$A_{G5}^y = L_{EF} \sin(\theta_{EF} + \beta_{EF} + \pi) \omega_{EF}^2 + L_{EF} \sin(\theta_{EF} + \beta_{EF} + \pi/2) \alpha_{EF} \quad (38)$$

$$A_{G6}^x = L_{GC} \cos(\theta_{GC} + \beta_{GC} + \pi) \omega_{GC}^2 + L_{GC} \cos(\theta_{GC} + \beta_{GC} + \pi/2) \alpha_{GC} \quad (39)$$

$$A_{G6}^y = L_{GC} \sin(\theta_{GC} + \beta_{GC} + \pi) \omega_{GC}^2 + L_{GC} \sin(\theta_{GC} + \beta_{GC} + \pi/2) \alpha_{GC} \quad (40)$$

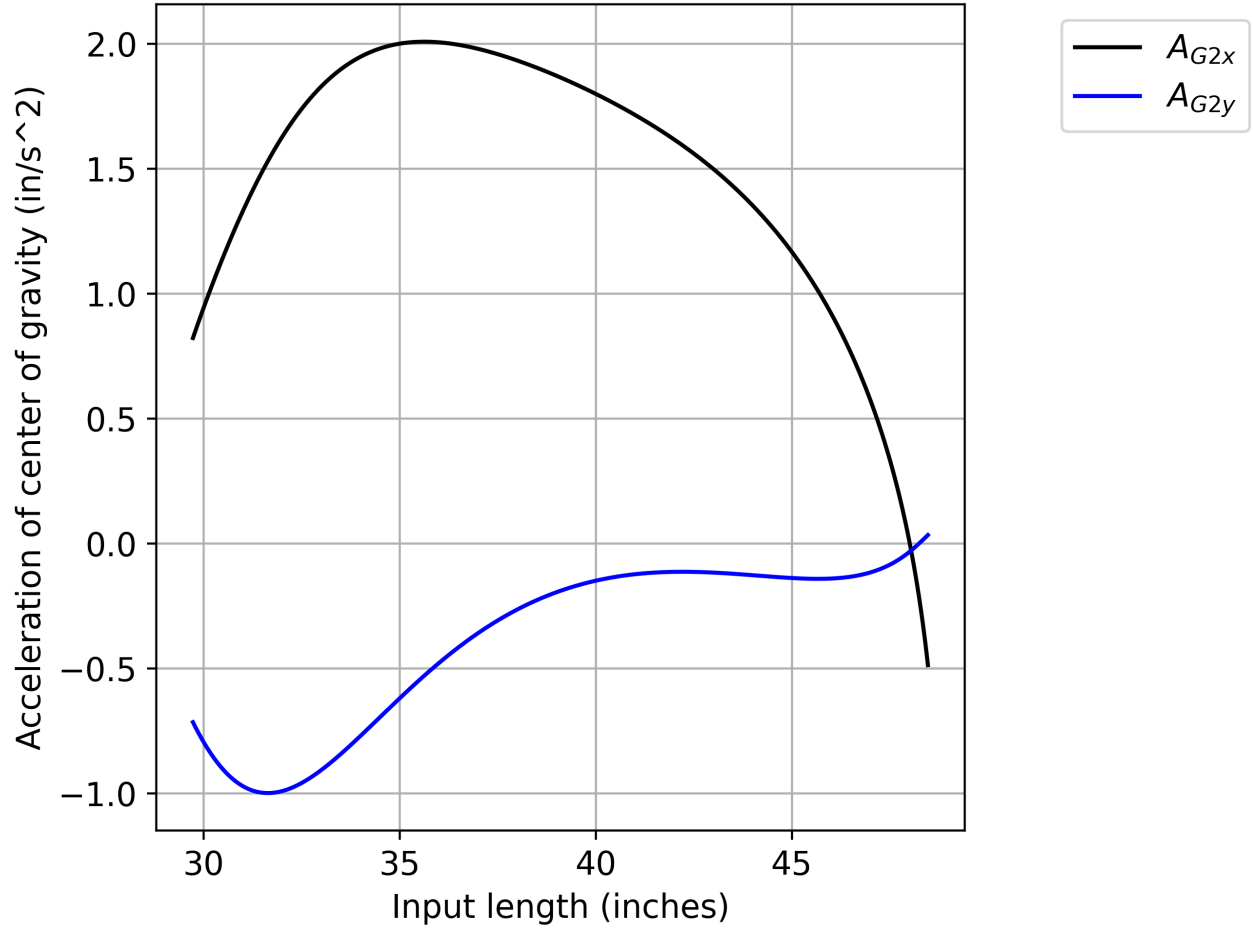


Figure 8: Acceleration of center of gravity of Link BA

The final part of the analysis is to write equations to solve for the forces and moments at each joint. The first step is to draw the free body diagrams of each link except for the ground link. Then using these, I wrote equations for the sum of forces and moments for each link. This gave me 15 equations and 15 unknowns for each angle once I set the output force. These equations can be seen below as Eqs. 41–55. The output and input forces as well as the positions of the center of inertia of each link can be seen in Figure 9. These equations are all linear equations, so because there are the same number of unknowns as equations, I used the NumPy linear solver function to solve these equations at each position. The plots of forces and moments I found can be seen in figs. 10 to 18.

Link Forces

$$F_{12}^x - F_{23}^x = m_2 A_{G2}^x \quad (41)$$

$$F_{12}^y - F_{23}^y = m_2 A_{G2}^y \quad (42)$$

$$F_{23}^x - F_{34}^x - F_{35}^x = m_3 A_{G3}^x \quad (43)$$

$$F_{23}^y - F_{34}^y - F_{35}^y - F_P = m_3 A_{G3}^y \quad (44)$$

$$F_{34}^x + F_i \cos(\theta_{DG}) - F_{46}^x \cos(\theta_{DG} + 90^\circ) = m_4 A_{G4}^x \quad (45)$$

$$F_{34}^y + F_i \sin(\theta_{DG}) - F_{46}^y \sin(\theta_{DG} + 90^\circ) = m_4 A_{G4}^y \quad (46)$$

$$F_{15}^x + F_{35}^x = m_5 A_{G5}^x \quad (47)$$

$$F_{15}^y + F_{35}^y = m_5 A_{G5}^y \quad (48)$$

$$F_{16}^x + F_{46}^x \cos(\theta_{DG} + 90^\circ) = m_6 A_{G6}^x \quad (49)$$

$$F_{16}^y + F_{46}^y \sin(\theta_{DG} + 90^\circ) = m_6 A_{G6}^y \quad (50)$$

$$\begin{aligned} & -F_{12}^x L_{BA} \sin(\theta_{BA} + 180^\circ) + F_{12}^y L_{BA} \cos(\theta_{BA} + 180^\circ) \\ & + F_{23}^x (R_{BA} - L_{BA}) \sin(\theta_{BA}) - F_{23}^y (R_{BA} - L_{BA}) \cos(\theta_{BA}) = I_{G2} \alpha_2 \end{aligned} \quad (51)$$

$$\begin{aligned} & -F_{23}^x L_{DB} \sin(\theta_{DB} + \beta_{DB} + 180^\circ) + F_{23}^y L_{DB} \cos(\theta_{DB} + \beta_{DB} + 180^\circ) \\ & - F_P (L_{DB} \cos(\theta_{DB} + \beta_{DB} + 180^\circ) + R_{DB} \cos(\theta_{DB} + \beta_B)) \\ & + F_{35}^x (L_{DB} \sin(\theta_{DB} + \beta_{DB} + 180^\circ) + R_{DB} \sin(\theta_{DB}) - R_{DE} \sin(\theta_{DE})) \\ & - F_{35}^y (L_{DB} \cos(\theta_{DB} + \beta_{DB} + 180^\circ) + R_{DB} \cos(\theta_{DB}) - R_{DE} \cos(\theta_{DE})) \\ & + F_{34}^x (L_{DB} \sin(\theta_{DB} + \beta_{DB} + 180^\circ) + R_{DB} \sin(\theta_{DB})) \\ & - F_{34}^y (L_{DB} \cos(\theta_{DB} + \beta_{DB} + 180^\circ) + R_{DB} \cos(\theta_{DB})) = I_{G3} \alpha_3 \end{aligned} \quad (52)$$

$$-M_{46} - F_{46} (L_4 - R_{DG}) - F_{34}^x L_4 \sin(\theta_{DG}) + F_{34}^y L_4 \cos(\theta_{DG}) = I_{G4} \alpha_4 \quad (53)$$

$$\begin{aligned} & -F_{35}^x (R_{EF} - L_{EF}) \sin(\theta_{EF}) + F_{35}^y (R_{EF} - L_{EF}) \cos(\theta_{EF}) \\ & - F_{15}^x L_{EF} \sin(\theta_{EF} + 180^\circ) + F_{15}^y L_{EF} \cos(\theta_{EF} + 180^\circ) = I_{G5} \alpha_5 \end{aligned} \quad (54)$$

$$M_{46} + F_{46} (R_{GC} - L_{GC}) - F_{16}^x L_{GC} \sin(\theta_{GC} + 180^\circ) + F_{16}^y L_{GC} \cos(\theta_{GC} + 180^\circ) = I_{G6} \alpha_6 \quad (55)$$

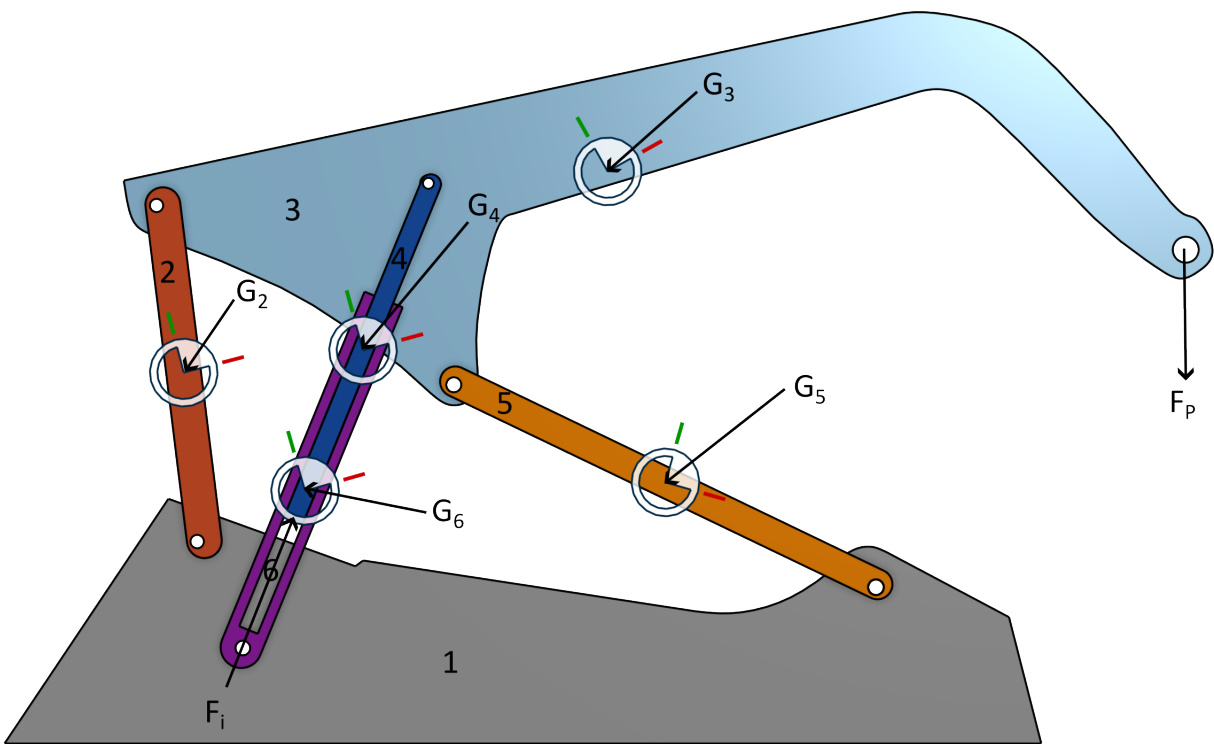


Figure 9: Centers of inertia and external forces on the linkage

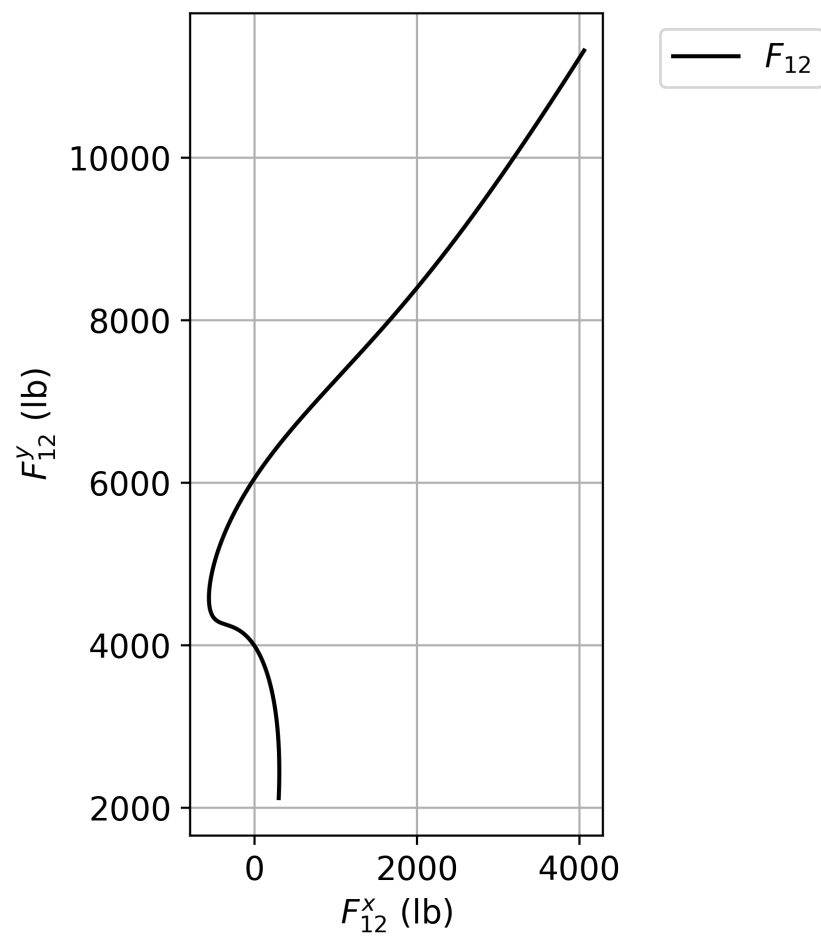


Figure 10: Plot of F_{12}

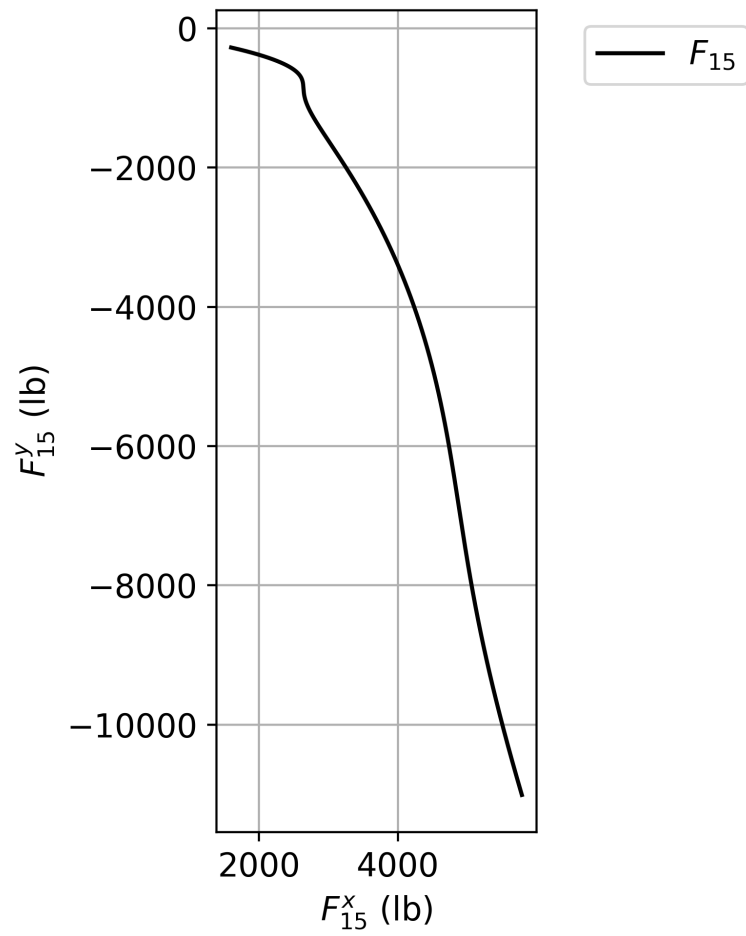


Figure 11: Plot of F_{15}

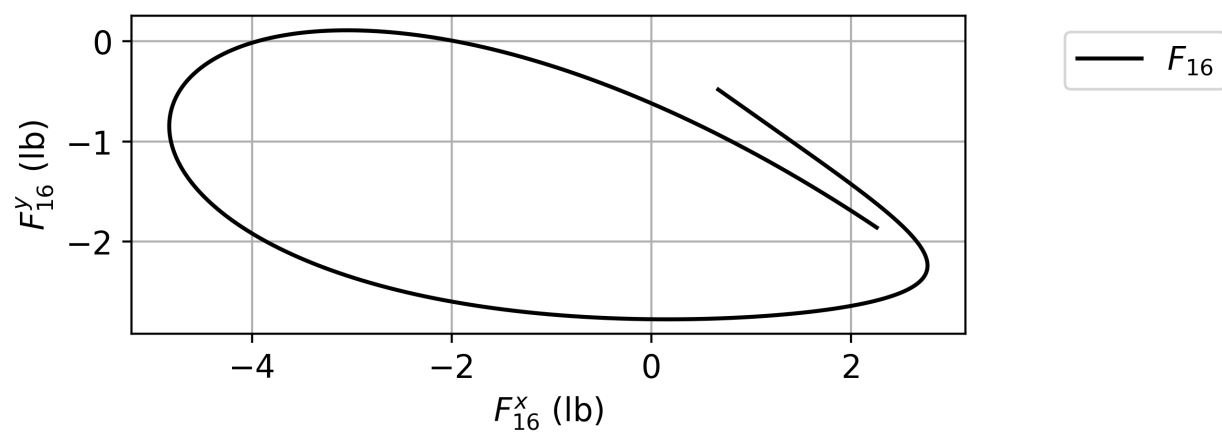


Figure 12: Plot of F_{16}

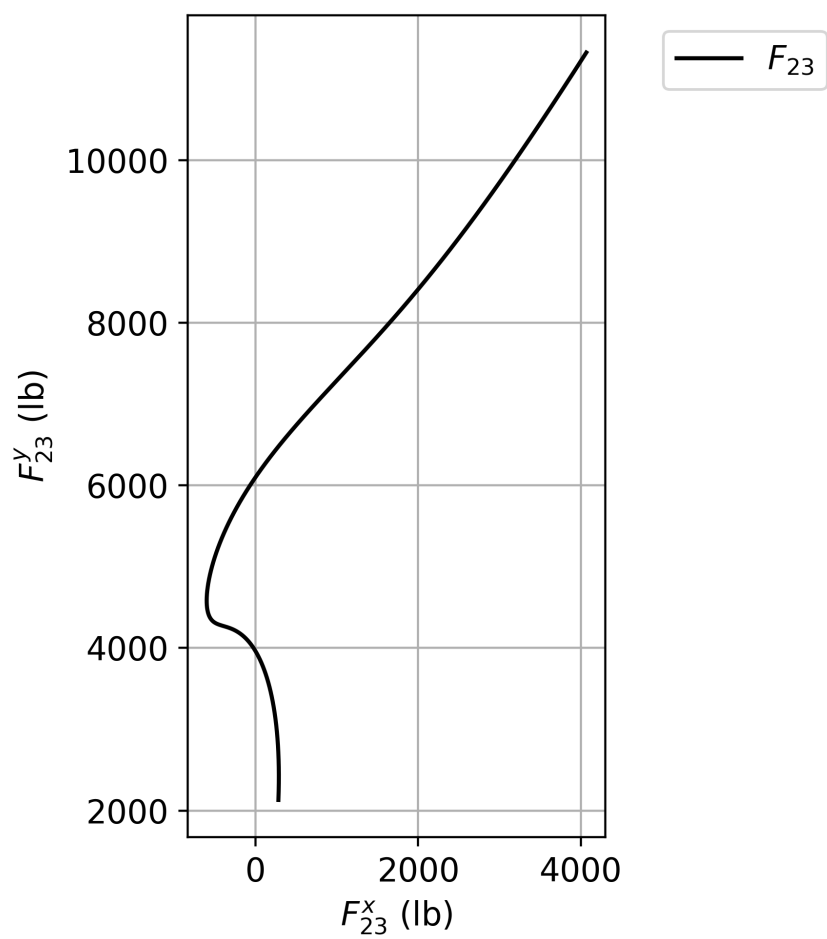


Figure 13: Plot of F_{23}

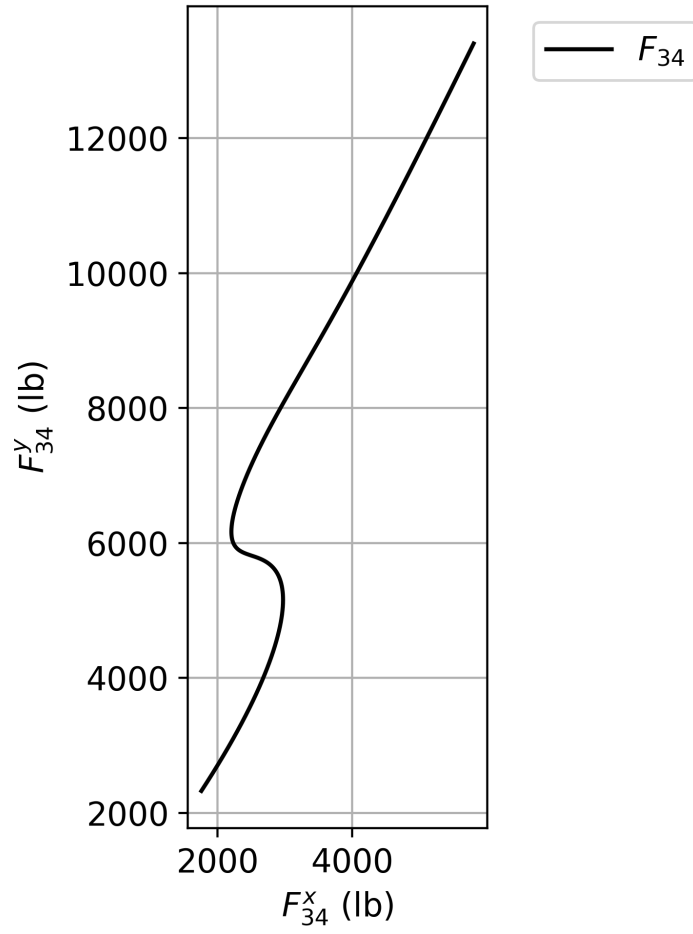


Figure 14: Plot of F_{34}

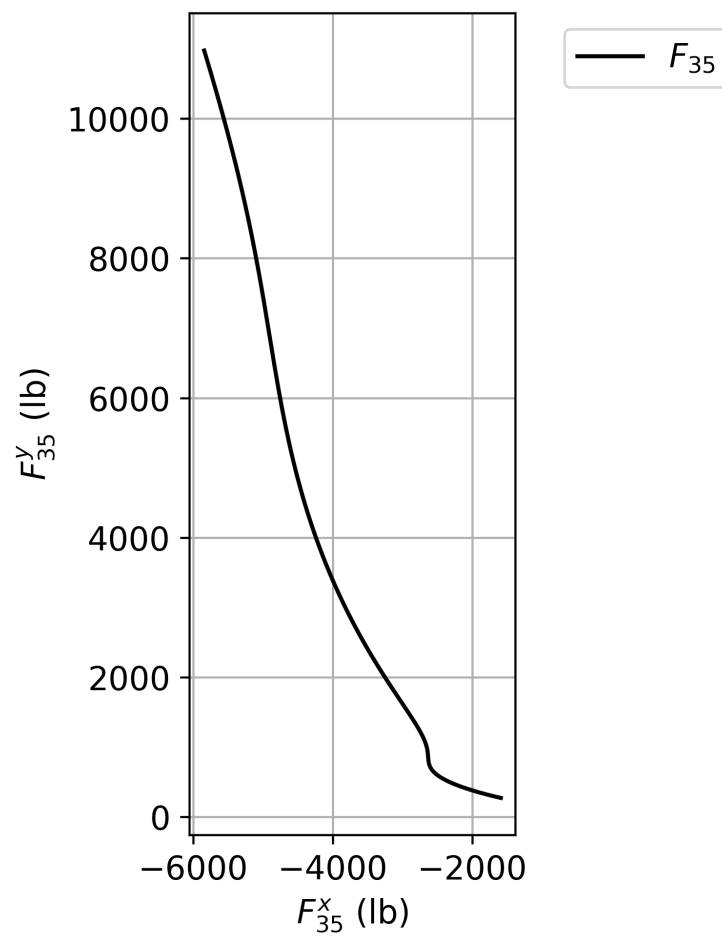


Figure 15: Plot of F_{35}

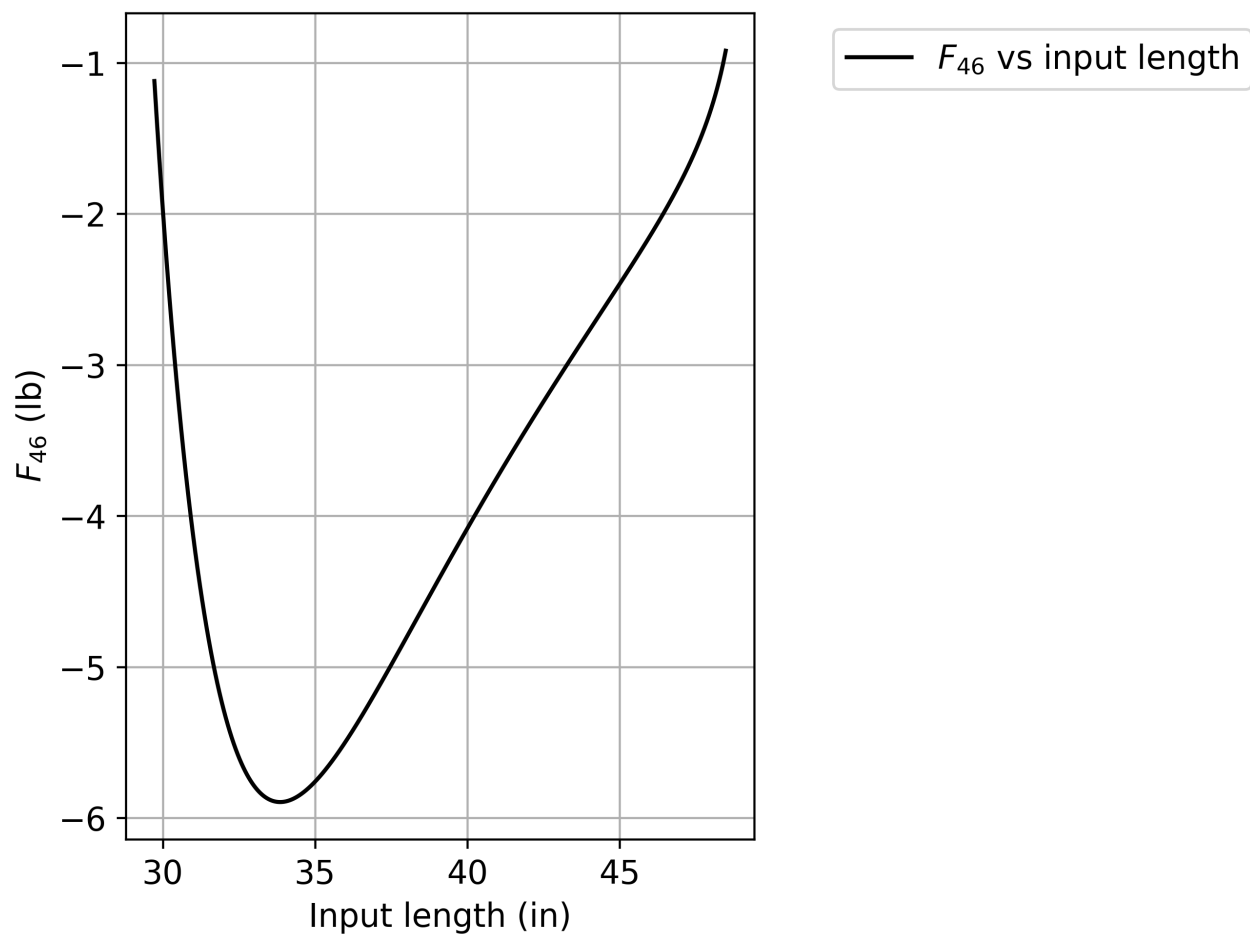


Figure 16: Plot of F_{46}

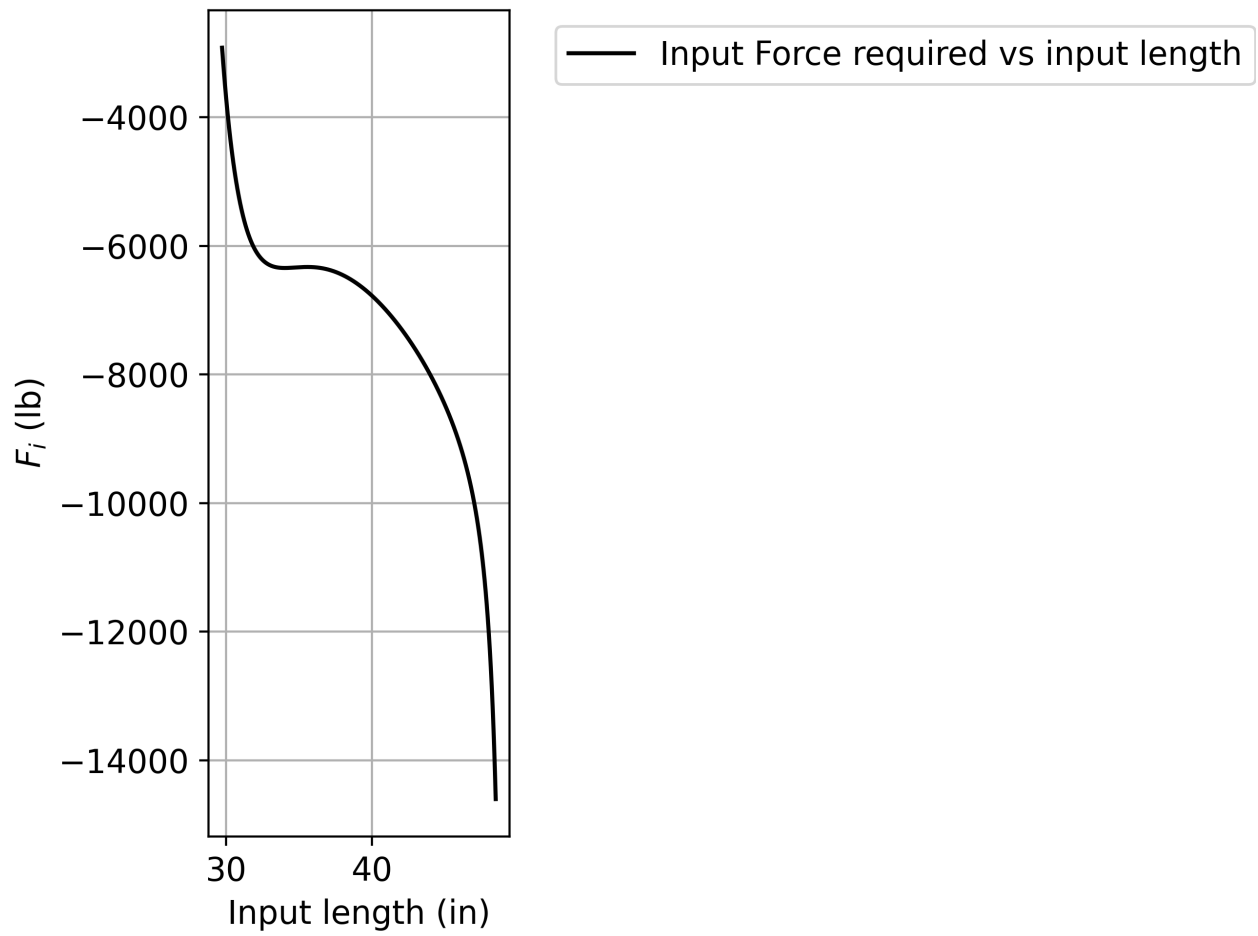


Figure 17: Plot of F_i

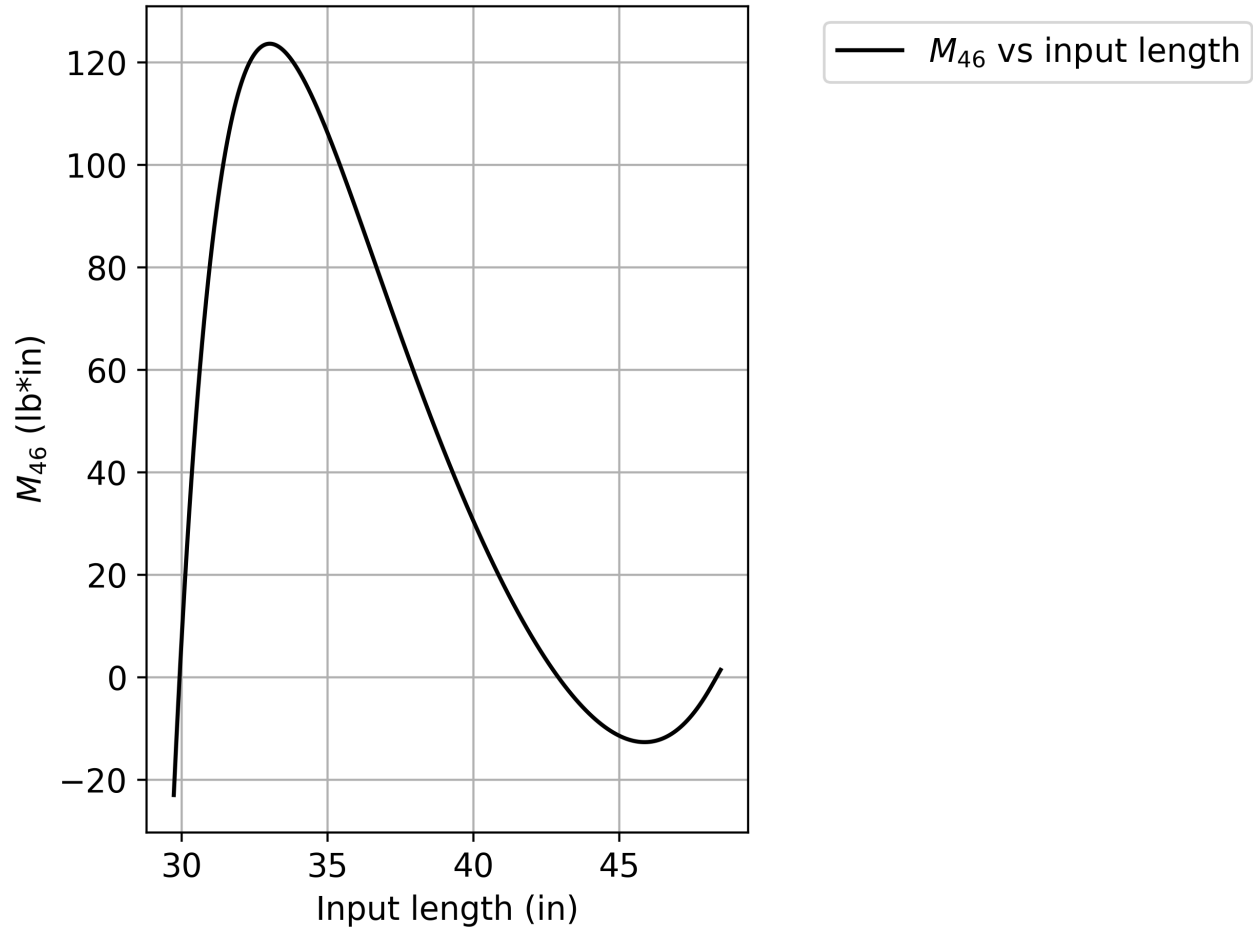


Figure 18: Plot of M_{46}

Link Measurements

$$R_{BA} = 24.4 \text{ in.}$$

$$R_{DB} = 19.8 \text{ in.}$$

$$R_{CA} = 8.5 \text{ in.}$$

$$\theta_{CA} = 293.04^\circ$$

$$R_{GC} = 25.5 \text{ in.}$$

$$R_{DE} = 14.7 \text{ in.}$$

$$R_{EF} = 33.9 \text{ in.}$$

$$R_{FC} = 45.9 \text{ in.}$$

$$\theta_{FC} = 5.44^\circ$$

$$\beta_D = 92.47^\circ$$

Conclusion

Linkages are an integral part of many mechanisms because they can provide mechanical advantage and move things along a non-straight path. Because the path is not simple however, the math and physics of the linkage become more complicated and most linkages beyond a four-bar linkage require using numerical methods to analyze the linkage. To use a numerical solver to analyze the linkage, you still need to know how to write the kinetic and kinematic equations of the linkage as well as how to solve these systems of equations with a solver. This project of analyzing a linkage on a skid-steer bucket loader helped me to practice these skills and discover difficulties in the process that are abstracted away when analyzing a linkage in a homework problem.

Multiloop Solver Code

```
1 import os
2 import numpy as np
3 import math
4 from scipy.optimize import fsolve
5 import matplotlib.pyplot as plt
6 from warnings import catch_warnings
7
8 act_img_ratio = 0.1718 # Approximate ratio of length without bucket from specs
9     ↳ in inches over length without bucket from image in mm
10
11 #act_img_ratio = 1      # Uncomment to use image link lengths instead of
12     ↳ approximated actual lengths
13 # Known values (For the moment just measured in millimeters from image
14     ↳ multiplied by ratio)
15 rba = 142.27*act_img_ratio
16 rdb = 115.2*act_img_ratio
17 rca = 49.46*act_img_ratio
18 thetaca = math.radians(293.04)
19 rgc = 148.5*act_img_ratio # Can't really tell from picture, but should not
20     ↳ affect results as long as accounted for accordingly
21 r4 = 148.5*act_img_ratio # Can't tell for this one either
22 rde = 85.65*act_img_ratio
23 ref = 197.12*act_img_ratio
24 rfc = 267.12*act_img_ratio
25 thetafc = math.radians(5.44) # angle from positive x-axis to ground link, which
26     ↳ is Rea
27 betad = math.radians(92.47) # Angle from DB to DA
28 rpb = 439.41*act_img_ratio # distance from point b to the bucket coupler point
29 betab = math.radians(-7.41) # Angle from rpb to rdb
30
31
32 # Centers of mass
33 # Lengths measured from second measured point in name
34 # Angles measured CCW from length vector
35 lba = rba/2
36 betaba = 0
37 #m2 = 100
38 m2=20
39 ig2 = m2*(3.530*10**(-5))/0.0068241
```

```

37
38 lgc = rgc/2 # Estimate
39 betagc = 0
40 #m6 = 100
41 m6=m2*0.0048194/0.0068241
42 ig6 = m6*(2.476*10**(-4))/0.0048194
43
44 l4 = r4/2 # Estimate
45 beta4 = 0
46 #m4 = 100
47 m4=m2*0.0040303/0.0068241
48 ig4 = m4*(1.654*10**(-4))/0.0040303
49
50 lef = ref/2
51 betaef = 0
52 #m5 = 100
53 m5=m2*0.0079326/0.0068241
54 ig5 = m5*(0.001476)/0.0079326
55
56 ldb = 0.036/0.022*rdb
57 betadb = math.radians(-0.2162)
58 #m3 = 200
59 m3=m2*0.3535955/0.0068241
60 ig3 = m3*(0.297341)/0.3535955
61
62
63
64 # Inputs
65 rdc = 173*act_img_ratio
66 rdotdg = 5 # rate of change of length of hydraulic piston (in/s)
67 rddotdg = 0 # rate of change of rate of change of length of hydraulic piston
    ↪ (in/s^2)
68 fp = 1000 # Weight on end of arm (lb)
69 rdg = rdc - rgc # Given input length
70
71
72 """
73 Unknowns:
74 x[0]: thetaba
75 x[1]: thetadb
76 x[2]: thetagc
77 x[3]: thetadg
78 x[4]: thetade
79 x[5]: thetaef
80 """
81 unknown_length_indices = []
82 used_inputs = []
83 num_equations = 6
84
85 # System of scalar equations to solve
86 def angle_eqs(x):
87     return [
88         rba*np.cos(x[0]) + rdb*np.cos(x[1]) - rca*np.cos(thetaca) -
            ↪ rgc*np.cos(x[2]) - rdg*np.cos(x[3]),

```

```

89     rba*np.sin(x[0]) + rdb*np.sin(x[1]) - rca*np.sin(thetaca) -
    ↪   rgc*np.sin(x[2]) - rdg*np.sin(x[3]),
90     rgc*np.cos(x[2]) + rdg*np.cos(x[3]) - rde*np.cos(x[4]) -
    ↪   ref*np.cos(x[5]) - rfc*np.cos(thetafc),
91     rgc*np.sin(x[2]) + rdg*np.sin(x[3]) - rde*np.sin(x[4]) -
    ↪   ref*np.sin(x[5]) - rfc*np.sin(thetafc),
92     x[2]-x[3],
93     x[1]+betad-x[4]
94 ]
95
96 def angular_velocity_eqs(thetaba, thetadb, thetagc, rdotdg, thetadg, thetade,
    ↪ thetaef, rdc):
97     rdg = rdc - rgc
98     return [
99         [-rba*np.sin(thetaba), -rdb*np.sin(thetadb), rgc*np.sin(thetagc),
    ↪   rdg*np.sin(thetadg), 0, 0],
100        [rba*np.cos(thetaba), rdb*np.cos(thetadb), -rgc*np.cos(thetagc),
    ↪   -rdg*np.cos(thetadg), 0, 0],
101        [0, 0, -rgc*np.sin(thetagc), -rdg*np.sin(thetadg), rde*np.sin(thetade),
    ↪   ref*np.sin(thetaef)],
102        [0, 0, rgc*np.cos(thetagc), rdg*np.cos(thetadg), -rde*np.cos(thetade),
    ↪   -ref*np.cos(thetaef)],
103        [0, 0, 1, -1, 0, 0],
104        [0, 1, 0, 0, -1, 0]
105    ], [
106        rdotdg*np.cos(thetadg),
107        -rdotdg*np.sin(thetadg),
108        -rdotdg*np.cos(thetadg),
109        -rdotdg*np.sin(thetadg),
110        0,
111        0
112    ]
113
114 def angular_acceleration_eqs(thetaba, thetadb, thetagc, rdotdg, thetadg,
    ↪ thetade, thetaef, rdc, rddotdg, omegaba, omegadb, omegagc, omegadg, omegade,
    ↪ omegaef):
115     rdg = rdc - rgc
116     return [
117         [-rba*np.sin(thetaba), -rdb*np.sin(thetadb), rgc*np.sin(thetagc),
    ↪   rdg*np.sin(thetadg), 0, 0],
118        [rba*np.cos(thetaba), -rdb*np.cos(thetadb), -rgc*np.cos(thetagc),
    ↪   -rdg*np.cos(thetadg), 0, 0],
119        [0, 0, -rgc*np.sin(thetagc), -rdg*np.sin(thetadg), rde*np.sin(thetade),
    ↪   ref*np.sin(thetaef)],
120        [0, 0, rgc*np.cos(thetagc), rdg*np.cos(thetadg), -rde*np.cos(thetade),
    ↪   -ref*np.cos(thetaef)],
121        [0, 0, 1, -1, 0, 0],
122        [0, 1, 0, 0, -1, 0]
123    ], [
124        rba*(omegaba**2)*np.cos(thetaba) + rdb*(omegadb**2)*np.cos(thetadb) -
    ↪   rgc*(omegagc**2)*np.cos(thetagc) + rddotdg*np.cos(thetadg) -
    ↪   2*rdotdg*omegadg*np.sin(thetadg) - rdg*(omegadg**2)*np.cos(thetadg),
125        rba*(omegaba**2)*np.sin(thetaba) + rdb*(omegadb**2)*np.sin(thetadb) -
    ↪   rgc*(omegagc**2)*np.sin(thetagc) + rddotdg*np.sin(thetadg) +
    ↪   2*rdotdg*omegadg*np.cos(thetadg) - rdg*(omegadg**2)*np.sin(thetadg),

```

```

126     rgc*(omegagc**2)*np.cos(thetagc) - rddotdg*np.cos(thetadg) +
    ↪ 2*rddotdg*omegadg*np.sin(thetadg) + rdg*(omegadg**2)*np.cos(thetadg)
    ↪ - rde*(omegade**2)*np.cos(thetade) -
    ↪ ref*(omegaef**2)*np.cos(thetaef),
127     rgc*(omegagc**2)*np.sin(thetagc) - rddotdg*np.sin(thetadg) -
    ↪ 2*rddotdg*omegadg*np.cos(thetadg) + rdg*(omegadg**2)*np.sin(thetadg)
    ↪ - rde*(omegade**2)*np.sin(thetade) -
    ↪ ref*(omegaef**2)*np.sin(thetaef),
128     0,
129     0
130 ]
131
132 def force_equations(thetaba, thetadb, thetagc, thetadg, thetade, thetaef, ag2x,
    ↪ ag2y, ag3x, ag3y, ag4x, ag4y, ag5x, ag5y, ag6x, ag6y, alpha2, alpha3,
    ↪ alpha4, alpha5, alpha6, rdc):
133     rdg = rdc - rgc
134     return [
135         [1, 0, 0, 0, 0, 0, -1, 0, 0, 0, 0, 0, 0, 0, 0],
136         [0, 1, 0, 0, 0, 0, 0, -1, 0, 0, 0, 0, 0, 0, 0],
137         [0, 0, 0, 0, 0, 0, 1, 0, -1, 0, -1, 0, 0, 0, 0],
138         [0, 0, 0, 0, 0, 0, 0, 1, 0, -1, 0, -1, 0, 0, 0],
139         [0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, -math.cos(thetadg + math.pi/2), 0,
    ↪ math.cos(thetadg)],
140         [0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, -math.sin(thetadg + math.pi/2), 0,
    ↪ math.sin(thetadg)],
141         [0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0],
142         [0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0],
143         [0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, math.cos(thetadg + math.pi/2), 0, 0],
144         [0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, math.sin(thetadg + math.pi/2), 0, 0],
145         [-lba*math.sin(thetaba + math.pi), lba*math.cos(thetaba + math.pi), 0, 0,
    ↪ 0, 0, (rba - lba)*math.sin(thetaba), -(rba - lba)*math.cos(thetaba),
    ↪ 0, 0, 0, 0, 0, 0, 0],
146         [0, 0, 0, 0, 0, 0, -ldb*math.sin(thetadb + betadb + math.pi),
    ↪ ldb*math.cos(thetadb + betadb + math.pi), (ldb*math.sin(thetadb +
    ↪ betadb + math.pi) + rdb*math.sin(thetadb)), -(ldb*math.cos(thetadb +
    ↪ betadb + math.pi) + rdb*math.cos(thetadb)), (ldb*math.sin(thetadb +
    ↪ betadb + math.pi) + rdb*math.sin(thetadb) - rde*math.sin(thetade)),
    ↪ -(ldb*math.cos(thetadb + betadb + math.pi) + rdb*math.cos(thetadb) -
    ↪ rde*math.cos(thetade)), 0, 0, 0],
147         [0, 0, 0, 0, 0, 0, 0, 0, -l4*math.sin(thetadg), l4*math.cos(thetadg), 0,
    ↪ 0, -(l4-rdg), -1, 0],
148         [0, 0, -lef*math.sin(thetaef + math.pi), lef*math.cos(thetaef + math.pi),
    ↪ 0, 0, 0, 0, 0, 0, -(ref - lef)*math.sin(thetaef),
    ↪ (ref-lef)*math.cos(thetaef), 0, 0, 0],
149         [0, 0, 0, 0, -lgc*math.sin(thetagc + math.pi), lgc*math.cos(thetagc +
    ↪ math.pi), 0, 0, 0, 0, 0, 0, (rgc - lgc), 1, 0]
150     ], [
151         m2*ag2x,
152         m2*ag2y,
153         m3*ag3x,
154         m3*ag3y + fp,
155         m4*ag4x,
156         m4*ag4y,
157         m5*ag5x,
158         m5*ag5y,

```

```

159         m6*ag6x,
160         m6*ag6y,
161         ig2*alpha2,
162         ig3*alpha3 + fp*(ldb*math.cos(thetadb + betadb + math.pi) +
        ↪ rdb*math.cos(thetadb + betab)),
163         ig4*alpha4,
164         ig5*alpha5,
165         ig6*alpha6
166     ]
167
168     # Rename pi for shorter call
169     pi = math.pi
170
171     # For finding right guesses
172     initial_guess_min = [0, 0, 0, 0, 0, 0]
173     initial_guess_max = [2*pi, 2*pi, 2*pi, 2*pi, 2*pi, 2*pi]
174     current_guess = [0, 0, 0, 0, 0, 0]
175
176     # Create empty set to store unique solutions
177     gen_solutions = set()
178
179     # Check if solution is in approximate range of motion
180     def result_check(result) -> bool:
181         if result[0] > 3*pi/4 or result[0] < pi/4:
182
183             return False
184         if result[1] > pi/2 and result[1] < 3*pi/2:
185
186             return False
187         if result[2] > pi/2:
188
189             return False
190         if result[3] > pi/2:
191
192             return False
193         if result[4] > pi:
194
195             return False
196         if result[5] < pi/2 or result[5] > pi:
197
198             return False
199         return True
200
201     # Check error of solution
202     """def calc_err(result, eq=eqs):
203         err = eq(result)
204         err_sum = 0
205         for i in range(len(err)):
206             err_sum += float(np.abs(err[i]))
207         return err_sum"""
208
209     def calc_err(result, eq=angle_eqs, unkn_len_ind=unknown_length_indices):
210         err = eq(result)
211         err_sum = 0
212         for i in range(len(err)):

```

```

213         if i in unkn_len_ind:
214             calc_err_amt = float(np.abs(err[i]))
215         else:
216             calc_err_amt = float(np.abs(err[i])) % 2*pi
217             calc_err_amt = min(calc_err_amt, 2*pi - calc_err_amt)
218         err_sum += calc_err_amt
219     return err_sum
220
221
222 # Populate the solution set with valid solutions
223 def solution_gen(current_guess: list[float] | None = None, guess_per_var: int =
    ↪ 5, index: int = 0, initial_min: list[float] = initial_guess_min,
    ↪ initial_max: list[float] = initial_guess_max, output: set[tuple[float, ...]]
    ↪ = gen_solutions, eq=angle_eqs, num_var: int = num_equations,
    ↪ unkn_len_ind=unknown_length_indices):
224     if current_guess is None:
225         current_guess = []
226     if len(initial_min) != num_var and len(initial_max) != num_var:
227         exit("Initial and/or final guess are wrong length")
228
229     if index == num_var:
230         with catch_warnings(record=True) as recorded_warnings:
231             sol = fsolve(eq, current_guess)
232             curr_error = calc_err(sol, eq)
233             if result_check(sol) and curr_error < 0.1:
234                 output.add(tuple(sol))
235         return
236
237     guesses = np.linspace(initial_min[index], initial_max[index], guess_per_var)
238
239     # Finds solutions with all different combinations of guesses
240     for guess in guesses:
241         solution_gen(current_guess=current_guess+[guess],
242             ↪ guess_per_var=guess_per_var, index=index+1, initial_min=initial_min,
243             ↪ initial_max=initial_max, output=output, eq=eq, num_var=num_var,
244             ↪ unkn_len_ind=unkn_len_ind)
245
246     return
247
248
249 # Absolute position of bucket coupler point with point C as the origin, positive
    ↪ x-axis to the right
250 def link_point_pos(angles):
251     point_c = (0, 0)
252
253     point_a = (-rca*math.cos(thetaca), -rca*math.sin(thetaca))
254
255     point_f = (rfc*math.cos(thetafc), rfc*math.sin(thetafc))
256
257     point_b = (point_a[0] + rba*math.cos(angles[0]), point_a[1] +
    ↪ rba*math.sin(angles[0]))
258
259     point_e = (point_f[0] + ref*math.cos(angles[5]), point_f[1] +
    ↪ ref*math.sin(angles[5]))

```

```

258
259     point_g = (rgc*math.cos(angles[2]), rgc*math.sin(angles[2]))
260
261     point_d = (point_e[0] + rde*math.cos(angles[4]), point_e[1] +
262         ↪ rde*math.sin(angles[4]))
263
264     bucket = (point_b[0] + rpb*math.cos(angles[1]+betab), point_b[1] +
265         ↪ rpb*math.sin(angles[1]+betab))
266
267     return point_a, point_b, point_c, point_d, point_e, point_f, point_g, bucket
268
269 # Generate solutions
270 solution_gen()
271
272 used_inputs.append(rdc)
273
274 # Turn set into a list so it is indexable
275 gen_solutions = list(gen_solutions)
276
277 min_error = 1
278 min_sol = None
279 for sol in gen_solutions:
280     err = calc_err(sol)
281     if err < min_error:
282         min_error = err
283         min_sol = sol
284
285
286 s = min_sol
287
288 if s is None:
289     exit("No solutions found")
290
291 sol_domain = np.linspace(rdc, 1.9*rgc, 10000)
292 rdc_store = rdc
293 sol_set = [s]
294 for x in sol_domain[1:]:
295     rdc = x
296     rdg = rdc - rgc
297     with catch_warnings(record=True) as record_warnings:
298         try:
299             new_sol = fsolve(angle_eqs, sol_set[-1])
300             sol_set.append(new_sol)
301             used_inputs.append(rdc)
302         except:
303             continue
304
305
306
307
308
309 rdc = rdc_store
310 rdg = rdc - rgc

```



```

311
312 point_ax = []
313 point_ay = []
314 point_bx = []
315 point_by = []
316 point_cx = []
317 point_cy = []
318 point_dx = []
319 point_dy = []
320 point_ex = []
321 point_ey = []
322 point_fx = []
323 point_fy = []
324 point_gx = []
325 point_gy = []
326 bucket_x = []
327 bucket_y = []
328
329 omegaba = []
330 omegadb = []
331 omegagc = []
332 omegadg = []
333 omegade = []
334 omegaef = []
335
336 alphaba = []
337 alphadb = []
338 alphagc = []
339 alphadg = []
340 alphade = []
341 alphaef = []
342 ag2x = []
343 ag2y = []
344 ag3x = []
345 ag3y = []
346 ag4x = []
347 ag4y = []
348 ag5x = []
349 ag5y = []
350 ag6x = []
351 ag6y = []
352
353
354 f12x = []
355 f12y = []
356 f15x = []
357 f15y = []
358 f16x = []
359 f16y = []
360 f23x = []
361 f23y = []
362 f34x = []
363 f34y = []
364 f35x = []
365 f35y = []

```

```

366 f46 = []
367 m46 = []
368 fi = []
369
370 ang_vel_sol_set = []
371 ang_acc_sol_set = []
372 for i in range(len(sol_set)):
373     point_pos = link_point_pos(sol_set[i])
374     point_ax.append(point_pos[0][0])
375     point_ay.append(point_pos[0][1])
376     point_bx.append(point_pos[1][0])
377     point_by.append(point_pos[1][1])
378     point_cx.append(point_pos[2][0])
379     point_cy.append(point_pos[2][1])
380     point_dx.append(point_pos[3][0])
381     point_dy.append(point_pos[3][1])
382     point_ex.append(point_pos[4][0])
383     point_ey.append(point_pos[4][1])
384     point_fx.append(point_pos[5][0])
385     point_fy.append(point_pos[5][1])
386     point_gx.append(point_pos[6][0])
387     point_gy.append(point_pos[6][1])
388     bucket_x.append(point_pos[7][0])
389     bucket_y.append(point_pos[7][1])
390
391     ang_vel_coeff, ang_vel_const = angular_velocity_eqs(sol_set[i][0],
392     ↪ sol_set[i][1], sol_set[i][2], rdotdg, sol_set[i][3], sol_set[i][4],
393     ↪ sol_set[i][5], used_inputs[i])
394     curr_ang_vel = np.linalg.solve(ang_vel_coeff, ang_vel_const)
395     ang_vel_sol_set.append(curr_ang_vel)
396     omegaba.append(curr_ang_vel[0])
397     omegadb.append(curr_ang_vel[1])
398     omegagc.append(curr_ang_vel[2])
399     omegadg.append(curr_ang_vel[3])
400     omegade.append(curr_ang_vel[4])
401     omegaef.append(curr_ang_vel[5])
402
403     ang_acc_coeff, ang_acc_const = angular_acceleration_eqs(sol_set[i][0],
404     ↪ sol_set[i][1], sol_set[i][2], rdotdg, sol_set[i][3], sol_set[i][4],
405     ↪ sol_set[i][5], used_inputs[i], rddotdg, ang_vel_sol_set[i][1],
406     ↪ ang_vel_sol_set[i][2], ang_vel_sol_set[i][3],
407     ↪ ang_vel_sol_set[i][4], ang_vel_sol_set[i][5])
408     curr_ang_acc = np.linalg.solve(ang_acc_coeff, ang_acc_const)
409     ang_acc_sol_set.append(curr_ang_acc)
410     alphaba.append(curr_ang_acc[0])
411     alphadb.append(curr_ang_acc[1])
412     alphagc.append(curr_ang_acc[2])
413     alphadg.append(curr_ang_acc[3])
414     alphade.append(curr_ang_acc[4])
415     alphaef.append(curr_ang_acc[5])
416
417     ag2x.append(lba*math.cos(sol_set[i][0] + betaba + math.pi) * (omegaba[-1]**2)
418     ↪ + lba*math.cos(sol_set[i][0] + betaba + (math.pi/2)) * alphaba[-1])
419     ag2y.append(lba*math.sin(sol_set[i][0] + betaba + math.pi) * (omegaba[-1]**2)
420     ↪ + lba*math.sin(sol_set[i][0] + betaba + (math.pi/2)) * alphaba[-1])

```

```

413 ag3x.append(rba*math.cos(sol_set[i][0] + math.pi) * (omegaba[-1]**2) +
    ↪ rba*math.cos(sol_set[i][0] + (math.pi/2)) * alphaba[-1] +
    ↪ ldb*math.cos(sol_set[i][1] + betadb + math.pi) * (omegadb[-1]**2) +
    ↪ ldb*math.cos(sol_set[i][1] + betadb + (math.pi/2)) * alphadb[-1])
414 ag3y.append(rba*math.sin(sol_set[i][0] + math.pi) * (omegaba[-1]**2) +
    ↪ rba*math.sin(sol_set[i][0] + (math.pi/2)) * alphaba[-1] +
    ↪ ldb*math.sin(sol_set[i][1] + betadb + math.pi) * (omegadb[-1]**2) +
    ↪ ldb*math.sin(sol_set[i][1] + betadb + (math.pi/2)) * alphadb[-1])
415 ag4x.append((rdc-r4)*math.cos(sol_set[i][2] + math.pi) * (omegagc[-1]**2) +
    ↪ (rdc-r4)*math.cos(sol_set[i][2] + (math.pi/2)) * alphagc[-1] +
    ↪ l4*math.cos(sol_set[i][3] + beta4 + math.pi) * (omegadg[-1]**2) +
    ↪ l4*math.cos(sol_set[i][3] + beta4 + (math.pi/2)) * alphadg[-1] +
    ↪ 2*rdotdg*math.cos(sol_set[i][3] + beta4 + (3*math.pi/2)) * omegadg[-1] +
    ↪ rddotdg*math.cos(sol_set[i][3] + beta4))
416 ag4y.append((rdc-r4)*math.sin(sol_set[i][2] + math.pi) * (omegagc[-1]**2) +
    ↪ (rdc-r4)*math.sin(sol_set[i][2] + (math.pi/2)) * alphagc[-1] +
    ↪ l4*math.sin(sol_set[i][3] + beta4 + math.pi) * (omegadg[-1]**2) +
    ↪ l4*math.sin(sol_set[i][3] + beta4 + (math.pi/2)) * alphadg[-1] +
    ↪ 2*rdotdg*math.sin(sol_set[i][3] + beta4 + (3*math.pi/2)) * omegadg[-1] +
    ↪ rddotdg*math.sin(sol_set[i][3] + beta4))
417 ag5x.append(lef*math.cos(sol_set[i][5] + betaef + math.pi) * (omegaef[-1]**2)
    ↪ + lef*math.cos(sol_set[i][5] + betaef + (math.pi/2)) * alphaef[-1])
418 ag5y.append(lef*math.sin(sol_set[i][5] + betaef + math.pi) * (omegaef[-1]**2)
    ↪ + lef*math.sin(sol_set[i][5] + betaef + (math.pi/2)) * alphaef[-1])
419 ag6x.append(lgc*math.cos(sol_set[i][2] + betagc + math.pi) * (omegagc[-1]**2)
    ↪ + lgc*math.cos(sol_set[i][2] + betagc + (math.pi/2)) * alphagc[-1])
420 ag6y.append(lgc*math.sin(sol_set[i][2] + betagc + math.pi) * (omegagc[-1]**2)
    ↪ + lgc*math.sin(sol_set[i][2] + betagc + (math.pi/2)) * alphagc[-1])
421
422 force_coeff, force_const = force_equations(sol_set[i][0], sol_set[i][1],
    ↪ sol_set[i][2], sol_set[i][3], sol_set[i][4], sol_set[i][5], ag2x[i],
    ↪ ag2y[i], ag3x[i], ag3y[i], ag4x[i], ag4y[i], ag5x[i], ag5y[i], ag6x[i],
    ↪ ag6y[i], alphaba[i], alphadb[i], alphadg[i], alphaef[i], alphagc[i],
    ↪ used_inputs[i])
423 current_forces = np.linalg.solve(force_coeff, force_const)
424 f12x.append(current_forces[0])
425 f12y.append(current_forces[1])
426 f15x.append(current_forces[2])
427 f15y.append(current_forces[3])
428 f16x.append(current_forces[4])
429 f16y.append(current_forces[5])
430 f23x.append(current_forces[6])
431 f23y.append(current_forces[7])
432 f34x.append(current_forces[8])
433 f34y.append(current_forces[9])
434 f35x.append(current_forces[10])
435 f35y.append(current_forces[11])
436 f46.append(current_forces[12])
437 m46.append(current_forces[13])
438 fi.append(current_forces[14])
439
440 def r_degrs(num):
441     return num/pi*180 % 360
442

```

```

443 s = list(s)
444 for i in range(len(s)):
445     if not i in unknown_length_indices:
446         s[i] = r_degrs(s[i])
447 print(f"Theta_BA = {round(s[0], 3)} degrees")
448 print(f"Theta_DB = {round(s[1], 3)} degrees")
449 print(f"Theta_GC = {round(s[2], 3)} degrees")
450 print(f"Theta_DG = {round(s[3], 3)} degrees")
451 print(f"Theta_DE = {round(s[4], 3)} degrees")
452 print(f"Theta_EF = {round(s[5], 3)} degrees")
453
454 plot_path =
    ↪ os.path.join(os.path.split(os.path.split(os.path.abspath(__file__))[0])[0],
    ↪ "imgs")
455
456 # Link Points Plot
457 plt.rcParams['font.size'] = 12
458 fig1, ax1 = plt.subplots(layout='constrained')
459 ax1.grid()
460 ax1.plot(bucket_x, bucket_y, 'black', label = 'Bucket Coupler Point')
461 ax1.plot(point_ax, point_ay, color='orange', marker='o', label = 'Point A')
462 ax1.plot(point_bx, point_by, 'red', label = 'Point B')
463 ax1.plot(point_cx, point_cy, color='green', marker='o', label = 'Point C')
464 ax1.plot(point_dx, point_dy, 'blue', label = 'Point D')
465 ax1.plot(point_ex, point_ey, 'purple', label = 'Point E')
466 ax1.plot(point_fx, point_fy, color='gray', marker='o', label = 'Point F')
467 ax1.plot(point_gx, point_gy, 'maroon', label = 'Point G')
468 ax1.set_xlim(-20, 80)
469 ax1.set_ylim(-20, 80)
470 ax1.set_aspect('equal', adjustable='box')
471 ax1.set_xlabel('Absolute x-coordinate (inches)')
472 ax1.set_ylabel('Absolute y-coordinate (inches)')
473 ax1.legend(bbox_to_anchor=(1.1, 1), loc='upper left')
474 plt.savefig(os.path.join(plot_path, "LinkPointCoordinates.png"),
    ↪ bbox_inches='tight', dpi=400)
475
476
477 plt.rcParams['font.size'] = 12
478 fig1, ax1 = plt.subplots(layout='constrained')
479 ax1.grid()
480 ax1.plot(used_inputs, omegaba, color='black', linestyle='solid', label =
    ↪ '$\\omega_{ba}$')
481 ax1.set_xlabel('Input length (inches)')
482 ax1.set_ylabel('Angular velocity (rad/s)')
483 ax1.legend(bbox_to_anchor=(1.1, 1), loc='upper left')
484 plt.savefig(os.path.join(plot_path, "LinkBAAngVel.png"), dpi=400)
485
486
487 plt.rcParams['font.size'] = 12
488 fig1, ax1 = plt.subplots(layout='constrained')
489 ax1.grid()
490 ax1.plot(used_inputs, alphaba, color='black', linestyle='solid', label =
    ↪ '$\\alpha_{ba}$')
491 ax1.set_xlabel('Input length (inches)')
492 ax1.set_ylabel('Angular acceleration (rad/s^2)')

```

```

493 ax1.legend(bbox_to_anchor=(1.1, 1), loc='upper left')
494 plt.savefig(os.path.join(plot_path, "LinkBAAngAcc.png"), dpi=400)
495
496 plt.rcParams['font.size'] = 12
497 fig1, ax1 = plt.subplots(layout='constrained')
498 ax1.grid()
499 ax1.plot(used_inputs, ag2x, color='black', linestyle='solid', label =
    ↳ '$A_{G2x}$')
500 ax1.plot(used_inputs, ag2y, color='blue', linestyle='solid', label =
    ↳ '$A_{G2y}$')
501 ax1.set_xlabel('Input length (inches)')
502 ax1.set_ylabel('Acceleration of center of gravity (in/s^2)')
503 ax1.legend(bbox_to_anchor=(1.1, 1), loc='upper left')
504 plt.savefig(os.path.join(plot_path, "LinkBAAccCG.png"), dpi=400)
505
506 plt.rcParams['font.size'] = 12
507 fig1, ax1 = plt.subplots(layout='constrained')
508 ax1.grid()
509 ax1.plot(ag2x, ag2y, color='black', linestyle='solid', label = '$A_{G2}$')
510 ax1.set_aspect('equal', adjustable='box')
511 ax1.set_xlabel('Acceleration of center of gravity (in/s^2)')
512 ax1.set_ylabel('Acceleration of center of gravity (in/s^2)')
513 ax1.legend(bbox_to_anchor=(1.1, 1), loc='upper left')
514 plt.savefig(os.path.join(plot_path, "LinkBAAccCGCombined.png"), dpi=400)
515
516 plt.rcParams['font.size'] = 12
517 fig1, ax1 = plt.subplots(layout='constrained')
518 ax1.grid()
519 ax1.plot(f12x, f12y, color='black', linestyle='solid', label = '$F_{12}$')
520 ax1.set_aspect('equal', adjustable='box')
521 ax1.set_xlabel('$F_{12}^x$ (lb)')
522 ax1.set_ylabel('$F_{12}^y$ (lb)')
523 ax1.legend(bbox_to_anchor=(1.1, 1), loc='upper left')
524 plt.savefig(os.path.join(plot_path, "Force12.png"), dpi=400)
525
526 plt.rcParams['font.size'] = 12
527 fig1, ax1 = plt.subplots(layout='constrained')
528 ax1.grid()
529 ax1.plot(f15x, f15y, color='black', linestyle='solid', label = '$F_{15}$')
530 ax1.set_aspect('equal', adjustable='box')
531 ax1.set_xlabel('$F_{15}^x$ (lb)')
532 ax1.set_ylabel('$F_{15}^y$ (lb)')
533 ax1.legend(bbox_to_anchor=(1.1, 1), loc='upper left')
534 plt.savefig(os.path.join(plot_path, "Force15.png"), dpi=400)
535
536 plt.rcParams['font.size'] = 12
537 fig1, ax1 = plt.subplots(layout='constrained')
538 ax1.grid()
539 ax1.plot(f16x, f16y, color='black', linestyle='solid', label = '$F_{16}$')
540 ax1.set_aspect('equal', adjustable='box')
541 ax1.set_xlabel('$F_{16}^x$ (lb)')
542 ax1.set_ylabel('$F_{16}^y$ (lb)')
543 ax1.legend(bbox_to_anchor=(1.1, 1), loc='upper left')
544 plt.savefig(os.path.join(plot_path, "Force16.png"), bbox_inches='tight',
    ↳ dpi=400)

```

```

545
546 plt.rcParams['font.size'] = 12
547 fig1, ax1 = plt.subplots(layout='constrained')
548 ax1.grid()
549 ax1.plot(f23x, f23y, color='black', linestyle='solid', label = '$F_{23}$')
550 ax1.set_aspect('equal', adjustable='box')
551 ax1.set_xlabel('$F_{23}^{x}$ (lb)')
552 ax1.set_ylabel('$F_{23}^{y}$ (lb)')
553 ax1.legend(bbox_to_anchor=(1.1, 1), loc='upper left')
554 plt.savefig(os.path.join(plot_path, "Force23.png"), dpi=400)
555
556 plt.rcParams['font.size'] = 12
557 fig1, ax1 = plt.subplots(layout='constrained')
558 ax1.grid()
559 ax1.plot(f34x, f34y, color='black', linestyle='solid', label = '$F_{34}$')
560 ax1.set_aspect('equal', adjustable='box')
561 ax1.set_xlabel('$F_{34}^{x}$ (lb)')
562 ax1.set_ylabel('$F_{34}^{y}$ (lb)')
563 ax1.legend(bbox_to_anchor=(1.1, 1), loc='upper left')
564 plt.savefig(os.path.join(plot_path, "Force34.png"), dpi=400)
565
566 plt.rcParams['font.size'] = 12
567 fig1, ax1 = plt.subplots(layout='constrained')
568 ax1.grid()
569 ax1.plot(f35x, f35y, color='black', linestyle='solid', label = '$F_{35}$')
570 ax1.set_aspect('equal', adjustable='box')
571 ax1.set_xlabel('$F_{35}^{x}$ (lb)')
572 ax1.set_ylabel('$F_{35}^{y}$ (lb)')
573 ax1.legend(bbox_to_anchor=(1.1, 1), loc='upper left')
574 plt.savefig(os.path.join(plot_path, "Force35.png"), dpi=400)
575
576 plt.rcParams['font.size'] = 12
577 fig1, ax1 = plt.subplots(layout='constrained')
578 ax1.grid()
579 ax1.plot(used_inputs, f46, color='black', linestyle='solid', label = '$F_{46}$
↪ vs input length')
580 ax1.set_xlabel('Input length (in)')
581 ax1.set_ylabel('$F_{46}$ (lb)')
582 ax1.legend(bbox_to_anchor=(1.1, 1), loc='upper left')
583 plt.savefig(os.path.join(plot_path, "Force46.png"), dpi=400)
584
585 plt.rcParams['font.size'] = 12
586 fig1, ax1 = plt.subplots(layout='constrained')
587 ax1.grid()
588 ax1.plot(used_inputs, m46, color='black', linestyle='solid', label = '$M_{46}$
↪ vs input length')
589 ax1.set_xlabel('Input length (in)')
590 ax1.set_ylabel('$M_{46}$ (lb*in)')
591 ax1.legend(bbox_to_anchor=(1.1, 1), loc='upper left')
592 plt.savefig(os.path.join(plot_path, "Moment46.png"), dpi=400)
593
594 plt.rcParams['font.size'] = 12
595 fig1, ax1 = plt.subplots(layout='constrained')
596 ax1.grid()

```

```
597 ax1.plot(used_inputs, fi, color='black', linestyle='solid', label = 'Input Force  
    ↪ required vs input length')
598 ax1.set_xlabel('Input length (in)')
599 ax1.set_ylabel('$F_{i}$ (lb)')
600 ax1.legend(bbox_to_anchor=(1.1, 1), loc='upper left')
601 plt.savefig(os.path.join(plot_path, "Forcei.png"), dpi=400)
```
